

1. Project Motivation

In autonomous driving, accurately interpreting visual information is essential for safe and efficient navigation. One important task is recognizing street house numbers, which helps identify locations, verify destinations, and support mapping in urban environments. This project develops a machine learning model using the Street View House Numbers (SVHN) dataset, which contains real-world images of digits captured from street scenes.

The SVHN dataset is well suited for this task because it reflects real driving challenges, including varying lighting conditions, background clutter, occlusions, and perspective changes. Compared to simpler datasets, it provides a more realistic benchmark for models intended for practical deployment.

Since this is a safety-relevant application, high accuracy and reliability are critical. Even small classification errors can lead to incorrect location identification or reduced reliability, so the model must demonstrate strong performance, robustness, and generalization.

To address this, the project applies deep learning techniques, particularly convolutional neural networks (CNNs), which are effective at capturing spatial features in images. A comparison with a baseline multilayer perceptron (MLP) is included to evaluate performance differences.

The overall goal is to build a reliable digit recognition system that contributes to the perception capabilities of modern autonomous vehicles.

2. Description of the Business Problem

The main objective of this project is to compare a baseline Multi-Layer Perceptron (MLP) with a CNN-based transfer learning model using pretrained ResNet50 on the SVHN dataset for use in a driverless car system. The comparison aims to determine which approach provides stronger recognition accuracy, generalization, and robustness under real-world visual conditions.

3. Description of the Dataset / Data Source

3.1 Description

Dataset: **Street View House Numbers (SVHN)**

- A real-world image dataset created for digit recognition using house numbers collected from Google Street View images.

- It is widely used in machine learning because it is more realistic and challenging than simpler datasets like MNIST, since the images come from natural street scenes
- For this project, Format 2 is used, which is the cropped digits format. In Format 2, each image is a cropped 32×32 RGB digit image, suitable for digit classification.
- Each image in Format 2 is a 32 × 32 color image (RGB). For the MLP baseline, each image is flattened into a 3072-dimensional vector. For the ResNet50 model, each image is resized to 224 × 224 during preprocessing to match the network input requirements.
- The dataset contains 10 classes, with 1 class for each digit. Digit '1' has label 1, digit '9' has label 9, and digit '0' has label 10.
- The complete dataset can be found on the Stanford UFLDL SVHN webpage: [The Street View House Numbers \(SVHN\) Dataset](#)

3.2 Dataset Structure

The dataset is divided into three main `.mat` files and organized within the `data\` folder, as outlined below:

- `train_32x32.mat` : This file includes 73,257 digits for training set.
- `test_32x32.mat` : This file includes 26,032 digits for testing set.
- `extra_32x32.mat` : This file includes 531,131 additional digits that can be used as extra training data, if needed.

3.3 Load and Analyze the Data

In this section, we begin by setting up the working environment and preparing the dataset for analysis. If running in Google Colab, the workflow includes mounting the Google Drive to access project files and datasets. Next, we define the project directory structure, organizing paths for `data/`, `plots/`, and `saved_models/` to ensure consistency and reproducibility. Once the environment is configured, the dataset is loaded and explored through initial analysis steps outlined in the **EDA in Section 4**.

```
Mounted at /content/drive
IN_COLAB: True
PROJECT_ROOT: /content/drive/MyDrive/tafe/src/svhn
SAVED_MODELS_DIR: /content/drive/MyDrive/tafe/src/svhn/saved_models
PLOTS_DIR: /content/drive/MyDrive/tafe/src/svhn/plots
DATA_DIR: /content/drive/MyDrive/tafe/src/svhn/data

['train_32x32.mat', 'test_32x32.mat', 'extra_32x32.mat']
```

4. Exploratory Data Analysis (EDA) and Data Preparation Process

4.1 Dataset Exploration

The SVHN dataset was analysed to understand class distribution, image characteristics, and pixel intensity patterns. The dataset contains 32×32 RGB images of digits with relatively balanced class distribution. Visual inspection shows significant variability in digit placement, scale, and background noise, motivating the use of convolutional architectures and data augmentation.

Dataset Loading and Inspection. The SVHN (Street View House Numbers) dataset is stored in .mat (MATLAB format) files, which contain structured data such as images and labels.

```
Train keys: dict_keys(['__header__', '__version__', '__globals__', 'X', 'y'])
Test keys: dict_keys(['__header__', '__version__', '__globals__', 'X', 'y'])
Extra keys: dict_keys(['__header__', '__version__', '__globals__', 'X', 'y'])
```

Each .mat file is loaded as a Python dictionary, and these are the keys inside it. `X` contains all image data which is in shape of `(32, 32, 3, N)` where `32 × 32` is the image size, `3` refers to the RGB channels and `N` to number of images. `y` consists of the labels `0-9`.

The raw image data (`X`) and labels (`y`) are then extracted and stored in separate variables for training and testing. We also flatten the labels so it's easier to work with in machine learning models and libraries.

Label correction. SVHN uses 10 to represent digit 0, so labels must be corrected by converting all 10 values to 0 before training models.

Fixing SVHN Image Format. SVHN images are stored with the sample dimension last, so we reshape the data by transposing it to match the standard input format required by deep learning models.

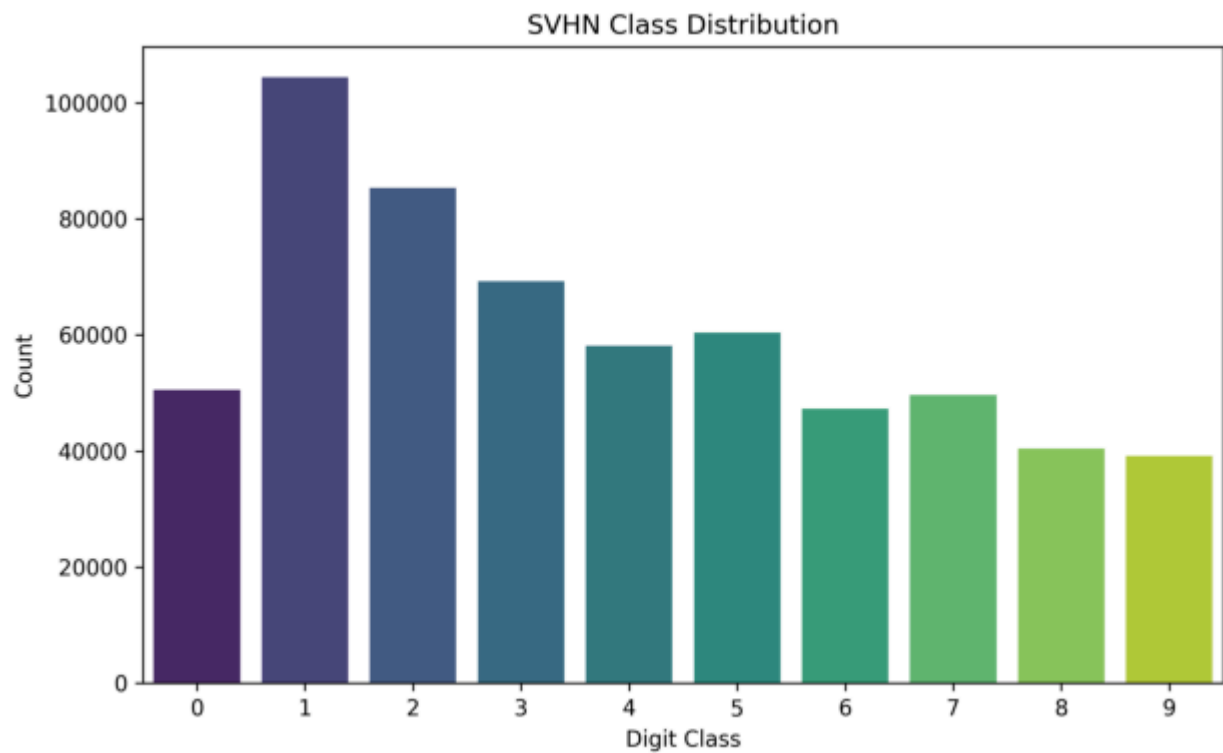
```
Train main shape: (73257, 32, 32, 3)
Extra shape: (531131, 32, 32, 3)
Test shape: (26032, 32, 32, 3)
```

Using all official training data plus a subset of the extra set balances improved model performance with reduced training time and computational cost.

```
Final training pool shape: (604388, 32, 32, 3)
Final training labels shape: (604388,)
Test shape: (26032, 32, 32, 3)
```

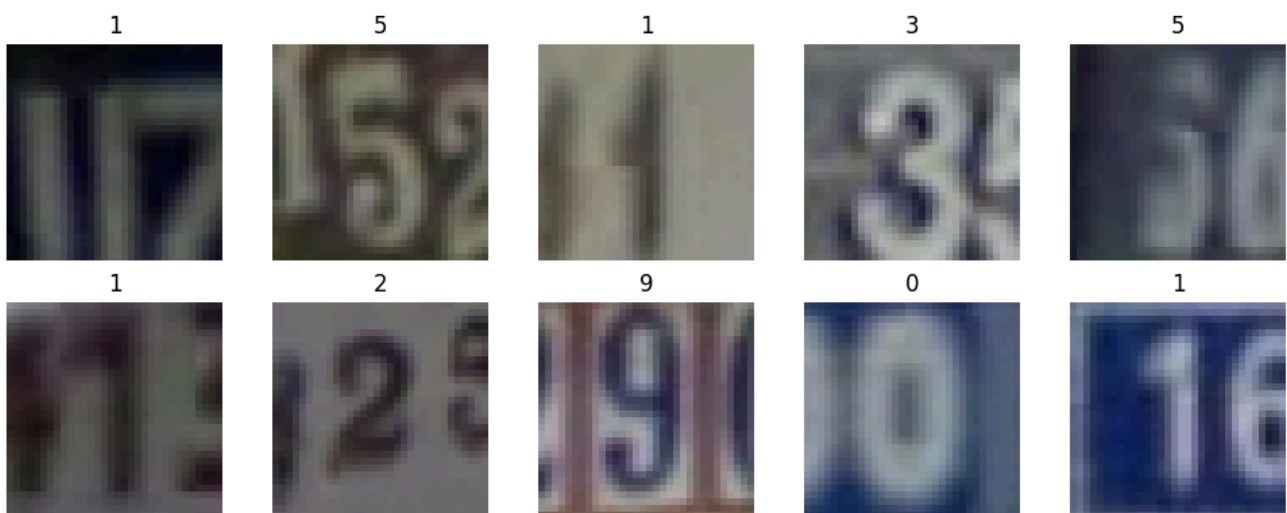
Class Distribution Analysis. The distribution of digit samples (0–9) in the SVHN dataset was examined to assess whether the dataset is balanced across all classes before training the model.

```
{np.uint8(0): np.int64(50498), np.uint8(1): np.int64(104421), np.uint8(2): np.int64(8532
5), np.uint8(3): np.int64(69262), np.uint8(4): np.int64(58091), np.uint8(5): np.int64(6037
2), np.uint8(6): np.int64(47309), np.uint8(7): np.int64(49592), np.uint8(8): np.int64(4040
3), np.uint8(9): np.int64(39115)}
Plot already exists at /content/drive/MyDrive/tafe/src/svhn/plots/svhn_class_distribution.
png. Loading it...
```

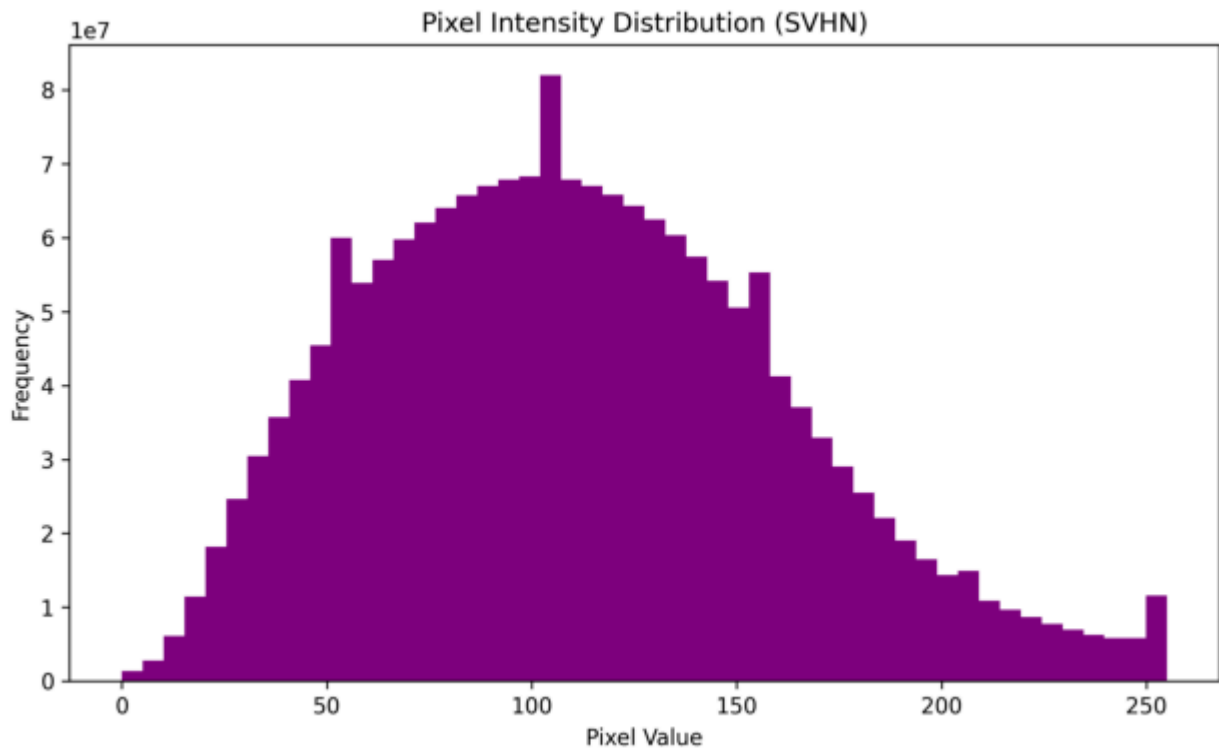


The chart above shows how many samples there are for each digit class (0-9) in the SVHN dataset. It can be seen that dataset is not perfectly balanced and some digits appear much more frequently than others. Example, digit 1 has the highest count (~105,000) samples while digits 8 and 9 have the lowest (~40,000) samples. Digits like 2, 3, 4, 5, 7 fall in the mid range (~50k-85k).

Below is grid of randomly selected SVHN images to visually inspect sample digits and verify the dataset's quality and diversity:



Plot already exists at `/content/drive/MyDrive/tafe/src/svhn/plots/pixel_distribution.png`.
Loading it...



The pixel intensity distribution above shows that most SVHN images contain pixel values in the mid-range, typically around 80 to 150, indicating that the images are generally moderately bright rather than extremely dark or bright. Very low pixel values (close to 0) and very high values (close to 255) occur less frequently, suggesting that extreme lighting conditions are rare in the dataset. The distribution follows a roughly bell-shaped pattern, which reflects a natural variation in illumination across images, with a slight skew toward higher intensity values, indicating the presence of some brighter regions. Overall, this balanced distribution is beneficial for model training, as it provides diverse yet stable input data and supports effective learning after normalization.

Plot already exists at `/content/drive/MyDrive/tafe/src/svhn/plots/mean_image.png`. Loading it...



The mean image above represents the average pixel values across all SVHN images, resulting in a blurred, gray image that reflects overall brightness and common spatial patterns rather than individual digits.

Plot already exists at `/content/drive/MyDrive/tafe/src/svhn/plots/std_image.png`. Loading it...

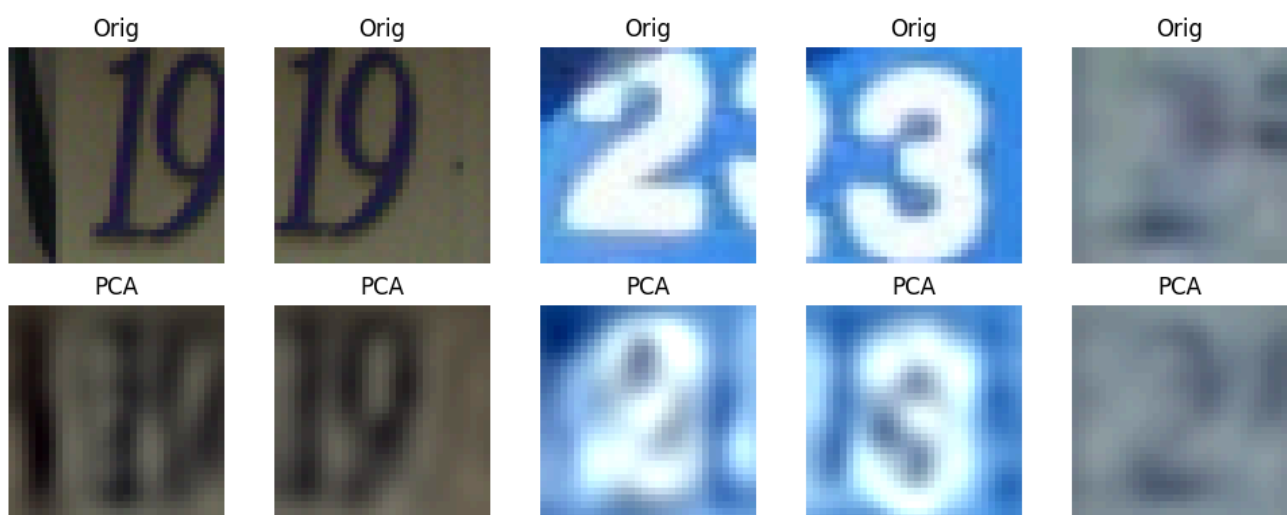


While, the SVHN Pixel Variability (Standard Deviation) Image above represents how much each pixel varies in intensity across all images. It shows that the central region of SVHN images has the highest variation (confirms that digits are generally centered), while outer regions are more consistent, indicating less informative background areas.

PCA was applied only as an exploratory data analysis technique to better understand the SVHN dataset, rather than as part of the actual model training process.

Shape after PCA: (5000, 50)

Explained variance ratio (first 10): [0.59053916 0.05579749 0.04904586 0.03992632 0.02057732 0.01840837 0.01550315 0.01367459 0.01249037 0.01117182]



The PCA results above show that the main shapes and digits are preserved, meaning the numbers remain visually identifiable after dimensionality reduction. However, there is an obvious loss of fine detail, visible in the blurrier and smoother PCA reconstructions. This happens because high-frequency image details are contained in later principal components that were discarded. Even so, the main colour patterns and contrast structure remain visible between the original and reconstructed images.

4.2 Data Pre-Processing

The dataset was preprocessed separately for the MLP and CNN models. A training-validation split was created from the sampled training set, while the original SVHN test set was kept untouched for final evaluation.

Train split: (543949, 32, 32, 3)
Validation split: (60439, 32, 32, 3)
Test: (26032, 32, 32, 3)

4.2.1 MLP Data Pre-Processing

This section prepares the SVHN image data for input into the Mixer-MLP model by:

- Standardizing pixel values using statistics computed from the training set only.
- Apply the same normalization to training, validation, and test data.
- Convert the processed data into PyTorch tensors.
- Ensure correct data types are used.
- Organize the data into TensorDatasets for training, validation, and testing.
- Prepare datasets in a format suitable for the Mixer-MLP training pipeline.

Mixer train shape: (543949, 32, 32, 3)
Mixer val shape: (60439, 32, 32, 3)
Mixer test shape: (26032, 32, 32, 3)

4.2.2 CNN Data Pre-Processing

This section prepares the SVHN image data for input into the ResNet50 model by:

- Load the default pretrained ResNet50 weights and extract the corresponding ImageNet normalization statistics.
- Define a training transform pipeline with resizing, data augmentation, tensor conversion, and normalization.
- Define an evaluation transform pipeline with resizing, tensor conversion, and normalization only.
- Use a custom SVHN dataset class so image transformations are applied dynamically during loading.
- Convert each image to uint8 format and return the corresponding label for training.
- Create a smaller training subset specifically for ResNet50 to reduce computational cost.
- Keep the validation and test sets unchanged for fair evaluation.
- Build separate training, validation, and test datasets using the appropriate transforms.

Original ResNet training size: 543949
Reduced ResNet training size: 435159

4.3 Device Setup

Hardware Configuration. Model training and evaluation were performed using PyTorch with automatic device selection. GPU acceleration was utilized when available to improve computational efficiency; otherwise, computations were executed on the CPU.

5. Methodology Used for Training, Testing, and Evaluation

To evaluate the effectiveness of different deep learning approaches for SVHN digit classification, two categories of models were investigated: a Mixer-MLP model representing multilayer perceptron-based architectures, and a convolutional neural network model based on ResNet50. The Mixer-MLP model was selected for its ability to model global spatial interactions without convolution, while the ResNet50 model was chosen for its strong capability to capture hierarchical spatial features and its proven performance in image classification tasks. To identify the most effective model configurations, a systematic hyperparameter search and training procedure was performed for each architecture. The process involved evaluating multiple hyperparameter combinations, monitoring validation performance, and selecting the best-performing model based on validation accuracy.

5.1 Model 1: MLP

A multilayer perceptron (MLP) is a type of neural network composed of fully connected layers that learns patterns by transforming input features through nonlinear activations, making it a simple yet effective baseline for classification tasks.

5.1.1 Mixer-MLP Architecture

Mixer-MLP is a neural network architecture that processes images by dividing them into patches and applying multilayer perceptrons (MLPs) to capture both spatial and channel-wise relationships, without using convolution or attention mechanisms. It is used for the SVHN dataset to efficiently model global interactions between image regions while maintaining a relatively simple and lightweight structure for digit classification.

5.1.2 Mixer-MLP Hyperparameters Tuning

Hyperparameter tuning was performed by testing several combinations of learning rate, batch size, dropout rate, number of epochs and hidden units. For each configuration, the model was trained and the best checkpoint based on validation accuracy was saved. The saved models were then compared, and the one with the highest validation accuracy was selected for final testing. **For full details on the hyperparameter search and model training, refer to Appendix B.**

5.1.3 Mixer-MLP Training

```
Global MLP experiment checkpoint found...
All 6 MLP experiments are already completed.
```

```
Skipping MLP hyperparameter search.
```

5.1.4 Mixer-MLP Evaluation

MLP Tuning Results:

	experiment	learning_rate	batch_size	dropout_rate	hidden_dim	\
0	2	0.0003	64	0.3	192	
1	1	0.0005	128	0.2	128	
2	4	0.0003	64	0.2	256	
3	6	0.0002	32	0.1	512	
4	5	0.0003	64	0.1	384	
5	3	0.0010	128	0.2	128	

	num_blocks	tokens_mlp_dim	channels_mlp_dim	num_epochs	best_val_acc	\
0	6	96	384	50	0.980245	
1	6	64	256	40	0.979583	
2	8	128	512	80	0.979517	
3	8	256	1024	100	0.979037	
4	6	192	768	80	0.978987	
5	4	64	256	30	0.978143	

	best_val_loss	best_epoch	\
0	0.095770	50	
1	0.090038	39	
2	0.112021	30	
3	0.215315	43	
4	0.177981	36	
5	0.094123	29	

```

                                checkpoint
0  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
1  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
2  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
3  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
4  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
5  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
Saved tuning results to /content/drive/MyDrive/tafe/src/svhn/saved_models/mlp_hyperparamet
er_results.csv

```

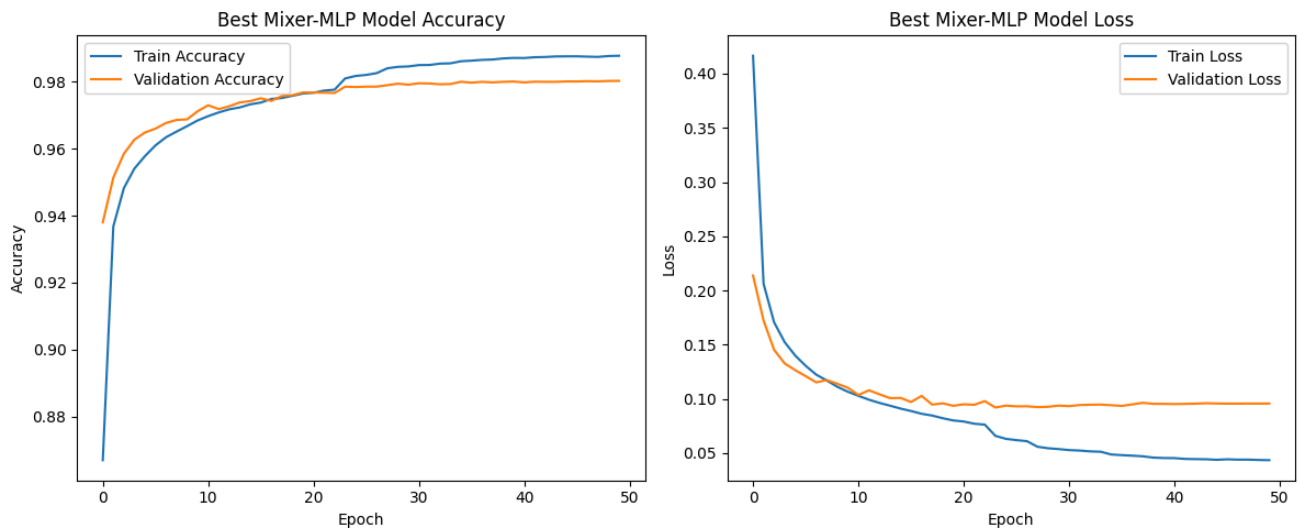
5.1.4.1 Load Best Mixer-MLP Model for Evaluation

Load the saved `.pth` checkpoint and set model to evaluation mode.

Best Mixer-MLP checkpoint selected: `/content/drive/MyDrive/tafe/src/svhn/saved_models/mixer_exp2_lr0_0003_bs64_drop0_3_hd192_b6_tm96_cm384.pth`

5.1.4.2 Training History Analysis of Best Mixer-MLP Model

These plots show the best model's training/validation loss accuracy curves.



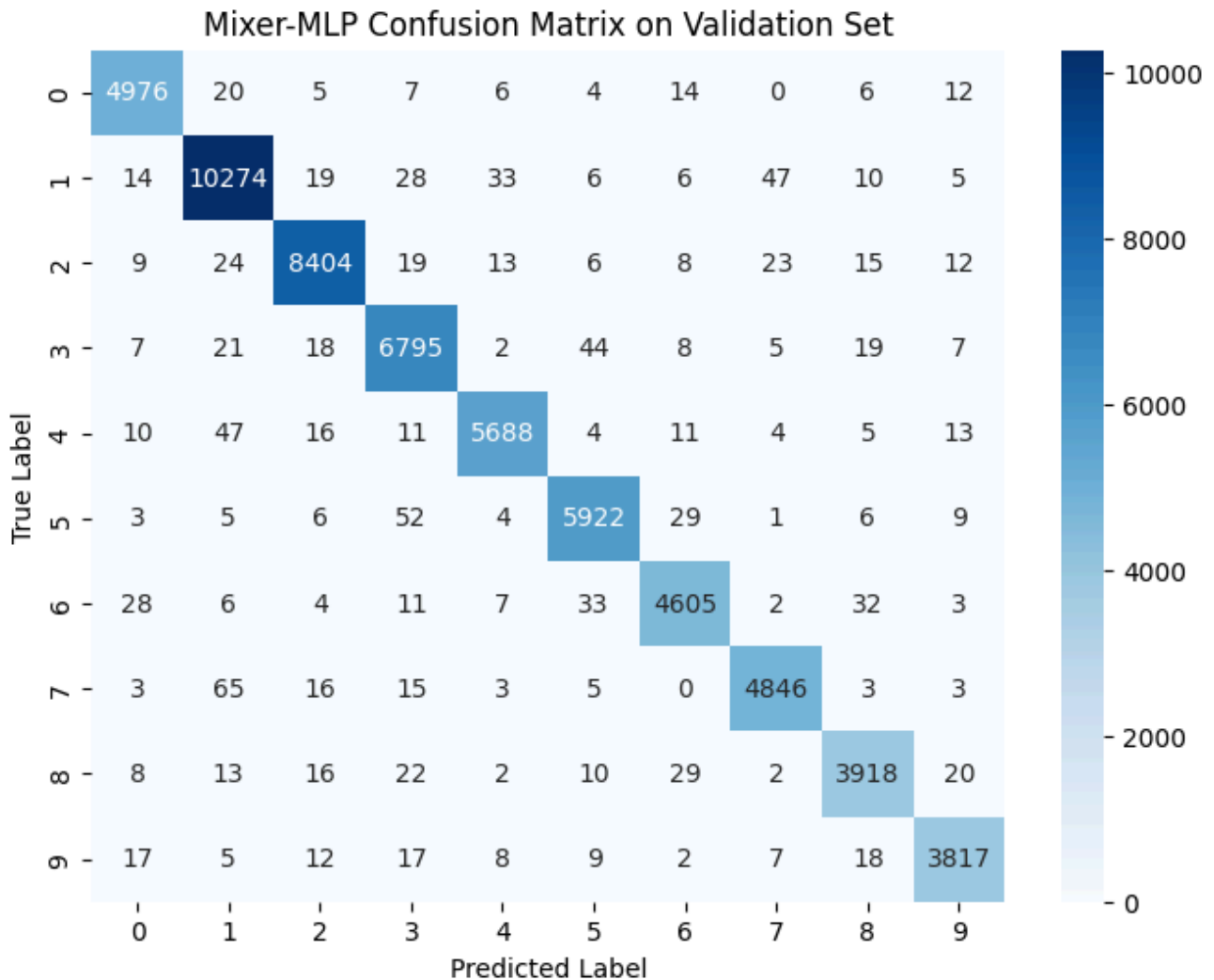
The performance of the best Mixer-MLP model over the training period demonstrates strong learning capability and stable convergence. As shown in the accuracy plot, the model achieves rapid improvement during the initial epochs, with training accuracy increasing sharply from approximately 87% to over 95% within a few iterations. Both training and validation accuracy continue to rise steadily, eventually reaching around 99% and 98%, respectively. The small gap between the two curves indicates that the model generalizes well to unseen data with minimal overfitting. Similarly, the loss curves show a significant reduction during the early stages of training, with both training and validation loss dropping quickly before gradually stabilizing. The training loss continues to decrease to a low value, while the validation loss plateaus at a slightly higher level, suggesting a minor degree of overfitting but overall stable performance. Overall, these results indicate that the Mixer-MLP model is able to efficiently learn discriminative features from the SVHN dataset, achieving high accuracy with good generalization and consistent training behavior.

5.1.4.3 Mixer-MLP Evaluation on Validation Dataset

Mixer-MLP Validation Accuracy: 0.9802

Mixer-MLP Classification Report on Validation Set:

	precision	recall	f1-score	support
0	0.98	0.99	0.98	5050
1	0.98	0.98	0.98	10442
2	0.99	0.98	0.99	8533
3	0.97	0.98	0.98	6926
4	0.99	0.98	0.98	5809
5	0.98	0.98	0.98	6037
6	0.98	0.97	0.98	4731
7	0.98	0.98	0.98	4959
8	0.97	0.97	0.97	4040
9	0.98	0.98	0.98	3912
accuracy			0.98	60439
macro avg	0.98	0.98	0.98	60439
weighted avg	0.98	0.98	0.98	60439



Per-Digit Accuracy of Mixer-MLP Model on Validation Set:

Digit 0: Accuracy = 98.53% (4976/5050)
 Digit 1: Accuracy = 98.39% (10274/10442)
 Digit 2: Accuracy = 98.49% (8404/8533)
 Digit 3: Accuracy = 98.11% (6795/6926)
 Digit 4: Accuracy = 97.92% (5688/5809)
 Digit 5: Accuracy = 98.10% (5922/6037)
 Digit 6: Accuracy = 97.34% (4605/4731)
 Digit 7: Accuracy = 97.72% (4846/4959)
 Digit 8: Accuracy = 96.98% (3918/4040)
 Digit 9: Accuracy = 97.57% (3817/3912)

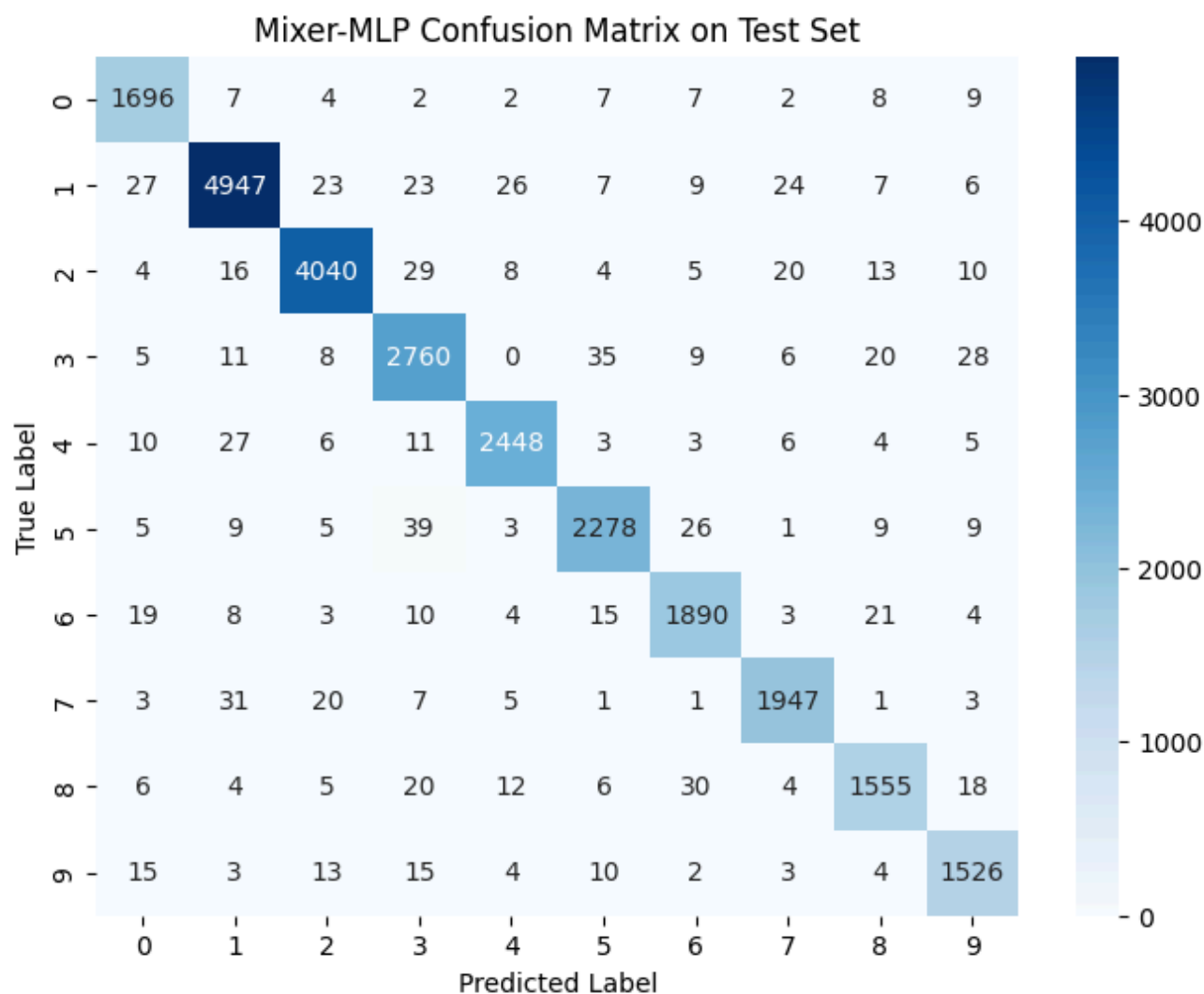
5.1.4.4 Mixer-MLP Evaluation on Test Dataset

After selecting the best-performing model based on validation performance, inference was performed on the unseen test dataset to assess the model's generalization capability. Final classification metrics, including accuracy, precision, recall, and F1-score, were computed to provide a comprehensive assessment of the model's effectiveness on previously unseen news articles.

Mixer-MLP Test Accuracy: 0.9637

Mixer-MLP Classification Report on Test Set:

	precision	recall	f1-score	support
0	0.95	0.97	0.96	1744
1	0.98	0.97	0.97	5099
2	0.98	0.97	0.98	4149
3	0.95	0.96	0.95	2882
4	0.97	0.97	0.97	2523
5	0.96	0.96	0.96	2384
6	0.95	0.96	0.95	1977
7	0.97	0.96	0.97	2019
8	0.95	0.94	0.94	1660
9	0.94	0.96	0.95	1595
accuracy			0.96	26032
macro avg	0.96	0.96	0.96	26032
weighted avg	0.96	0.96	0.96	26032



The confusion matrix above shows a strong concentration along the diagonal, indicating that the model achieves high classification accuracy across all digits. Most predictions match the true labels, confirming that the model generalizes well to unseen test data. Several digits are classified with particularly high accuracy, as indicated by large diagonal values, examples:

- Digit 1: 4,947 correct predictions (highest)

- Digit 2: 4,040 correct
- Digit 3: 2,760 correct
- Digit 5: 2,278 correct
- Digit 4: 2,448 correct

These results suggest that these digits have distinct visual features, making them easier for the model to identify. Some digits, while still performing well, show slightly higher misclassification rates:

- Digit 8: 1,555 correct, with confusion toward 6 (30 cases)
- Digit 9: 1,526 correct, with confusion toward 0 and 3
- Digit 7: 1,947 correct, with confusion toward 1 (31 cases)

The off-diagonal entries highlight specific digit pairs that are more frequently confused:

- 1 ↔ 7:
 - 31 instances where 7 is predicted as 1
 - 24 instances where 1 is predicted as 7
- 3 ↔ 5:
 - 35 instances of 3 → 5
 - 39 instances of 5 → 3
- 8 ↔ 6:
 - 30 instances of 8 → 6
- 4 ↔ 1:
 - 27 instances of 4 → 1

Below is the accuracy of Mixer-MLP Model on every digit on the test set:

Per-Digit Accuracy of Mixer-MLP Model on Test Set:

Digit 0: Accuracy = 97.25% (1696/1744)
 Digit 1: Accuracy = 97.02% (4947/5099)
 Digit 2: Accuracy = 97.37% (4040/4149)
 Digit 3: Accuracy = 95.77% (2760/2882)
 Digit 4: Accuracy = 97.03% (2448/2523)
 Digit 5: Accuracy = 95.55% (2278/2384)
 Digit 6: Accuracy = 95.60% (1890/1977)
 Digit 7: Accuracy = 96.43% (1947/2019)
 Digit 8: Accuracy = 93.67% (1555/1660)
 Digit 9: Accuracy = 95.67% (1526/1595)

5.2 Model 2: CNN

A convolutional neural network (CNN) is a type of deep learning model that uses convolutional layers to automatically learn spatial features from images, making it highly effective for tasks like digit classification in datasets such as SVHN.

5.2.1 ResNet50 Architecture

The final CNN model uses a pretrained ResNet50, which leverages residual connections and ImageNet-trained features to effectively learn complex spatial patterns. The model was adapted for

SVHN by resizing images to 224×224 and replacing the final layer with a 10-class classifier for digit recognition.

5.2.2 ResNet50 Hyperparameters Tuning

Hyperparameter tuning was performed by testing several combinations of learning rate, batch size, dropout rate, and number of epochs. For each configuration, the model was trained and the best checkpoint based on validation accuracy was saved. The saved models were then compared, and the one with the highest validation accuracy was selected for final testing. **For full details on the hyperparameter search and model training, refer to Appendix B.**

```
Global ResNet50 experiment checkpoint found...
All 6 ResNet50 experiments are already completed.
```

```
Skipping ResNet50 hyperparameter search.
```

5.2.3 ResNet50 Evaluation

Compare all saved models based on validation accuracy/loss and selecting the best-performing model.

```
ResNet50 Hyperparameter Tuning Results:
```

	experiment	learning_rate	batch_size	dropout_rate	num_epochs	\
0	1	0.00010	128	0.1	60	
1	5	0.00020	128	0.1	50	
2	2	0.00005	64	0.1	70	
3	4	0.00010	256	0.1	50	
4	3	0.00001	64	0.0	80	
5	6	0.00010	128	0.2	40	

	freeze_backbone	best_val_acc	best_val_loss	best_epoch	\
0	False	0.988600	0.051228	8	
1	False	0.988253	0.049822	11	
2	False	0.987905	0.065474	11	
3	False	0.987839	0.050237	6	
4	False	0.986747	0.081947	14	
5	True	0.985324	0.105475	38	

```
                                checkpoint
0  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
1  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
2  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
3  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
4  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
5  /content/drive/MyDrive/tafe/src/svhn/saved_mod...
Saved tuning results to /content/drive/MyDrive/tafe/src/svhn/saved_models/resnet50_hyperpa
rameter_results.csv
```

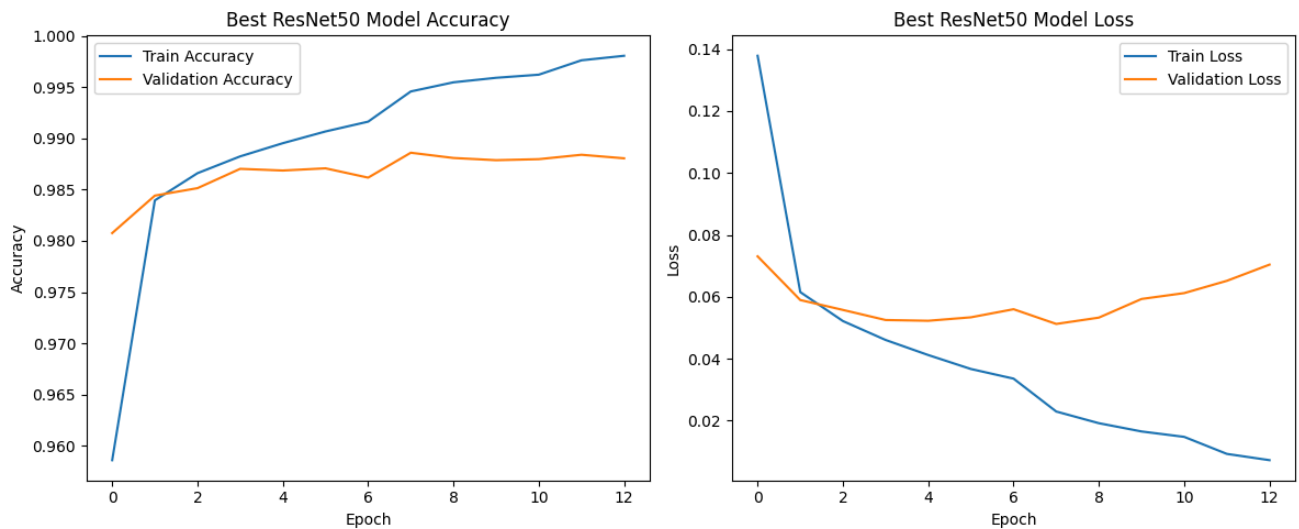
5.2.3.1 Load Best ResNet50 Model for Evaluation

Load the saved `.pth` checkpoint and set model to evaluation mode.

```
Best ResNet50 checkpoint selected: /content/drive/MyDrive/tafe/src/svhn/saved_models/resne
t50_exp1_lr0_0001_bs128_drop0_1_unfrozen.pth
```

5.2.3.2 Training History Analysis of Best ResNet50 Model

These plots show the best model's training/validation loss accuracy curves.



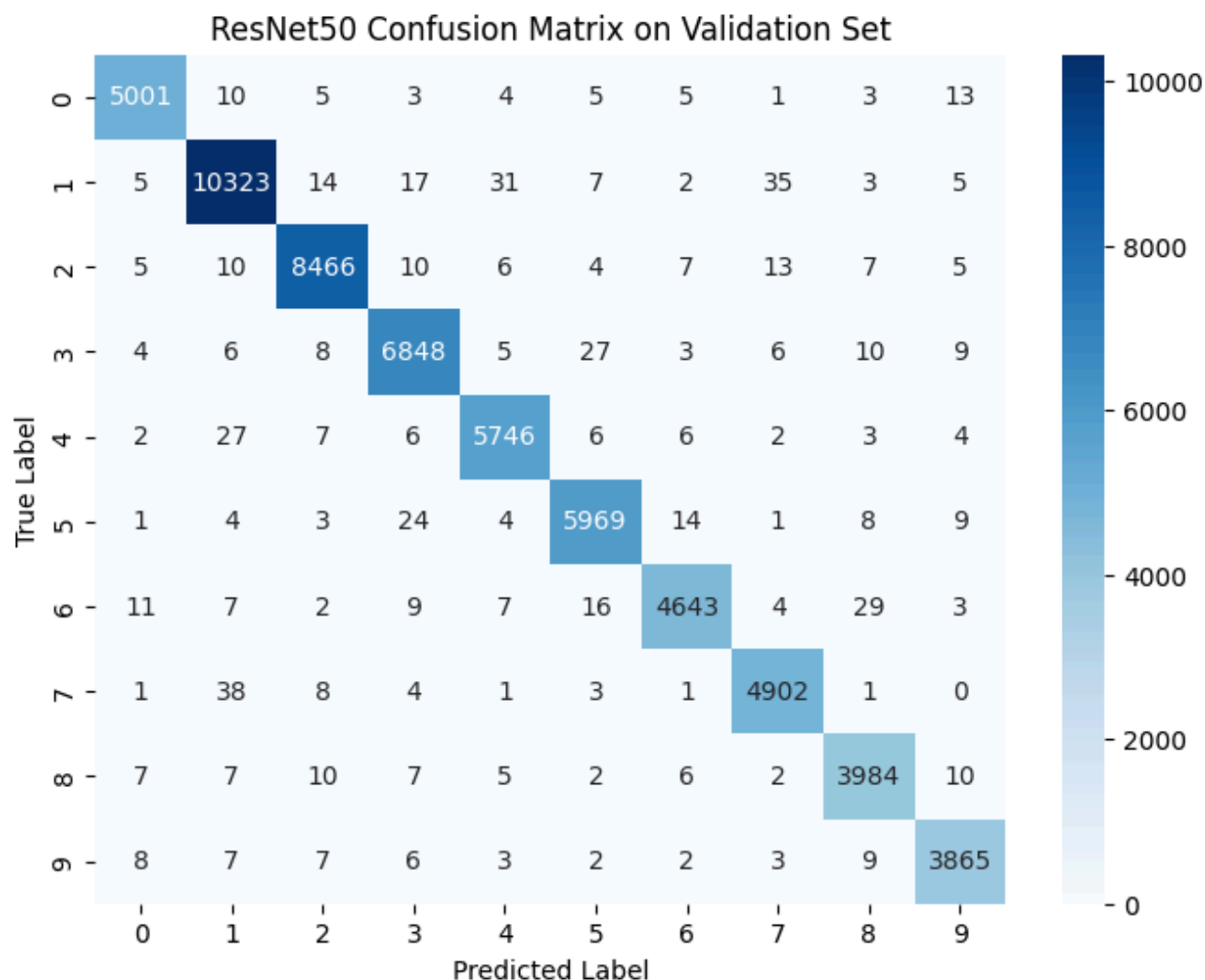
5.2.3.3 ResNet50 Evaluation on Validation Data

Using relevant metrics such as precision, recall, F1-score, and accuracy and Confusion Matrix visualization.

ResNet50 Validation Accuracy: 0.9886

ResNet50 Classification Report on Validation Set:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	5050
1	0.99	0.99	0.99	10442
2	0.99	0.99	0.99	8533
3	0.99	0.99	0.99	6926
4	0.99	0.99	0.99	5809
5	0.99	0.99	0.99	6037
6	0.99	0.98	0.99	4731
7	0.99	0.99	0.99	4959
8	0.98	0.99	0.98	4040
9	0.99	0.99	0.99	3912
accuracy			0.99	60439
macro avg	0.99	0.99	0.99	60439
weighted avg	0.99	0.99	0.99	60439



Per-Digit Accuracy of ResNet50 Model on Validation Set:

```
-----
Digit 0: Accuracy = 99.03% (5001/5050)
Digit 1: Accuracy = 98.86% (10323/10442)
Digit 2: Accuracy = 99.21% (8466/8533)
Digit 3: Accuracy = 98.87% (6848/6926)
Digit 4: Accuracy = 98.92% (5746/5809)
Digit 5: Accuracy = 98.87% (5969/6037)
Digit 6: Accuracy = 98.14% (4643/4731)
Digit 7: Accuracy = 98.85% (4902/4959)
Digit 8: Accuracy = 98.61% (3984/4040)
Digit 9: Accuracy = 98.80% (3865/3912)
```

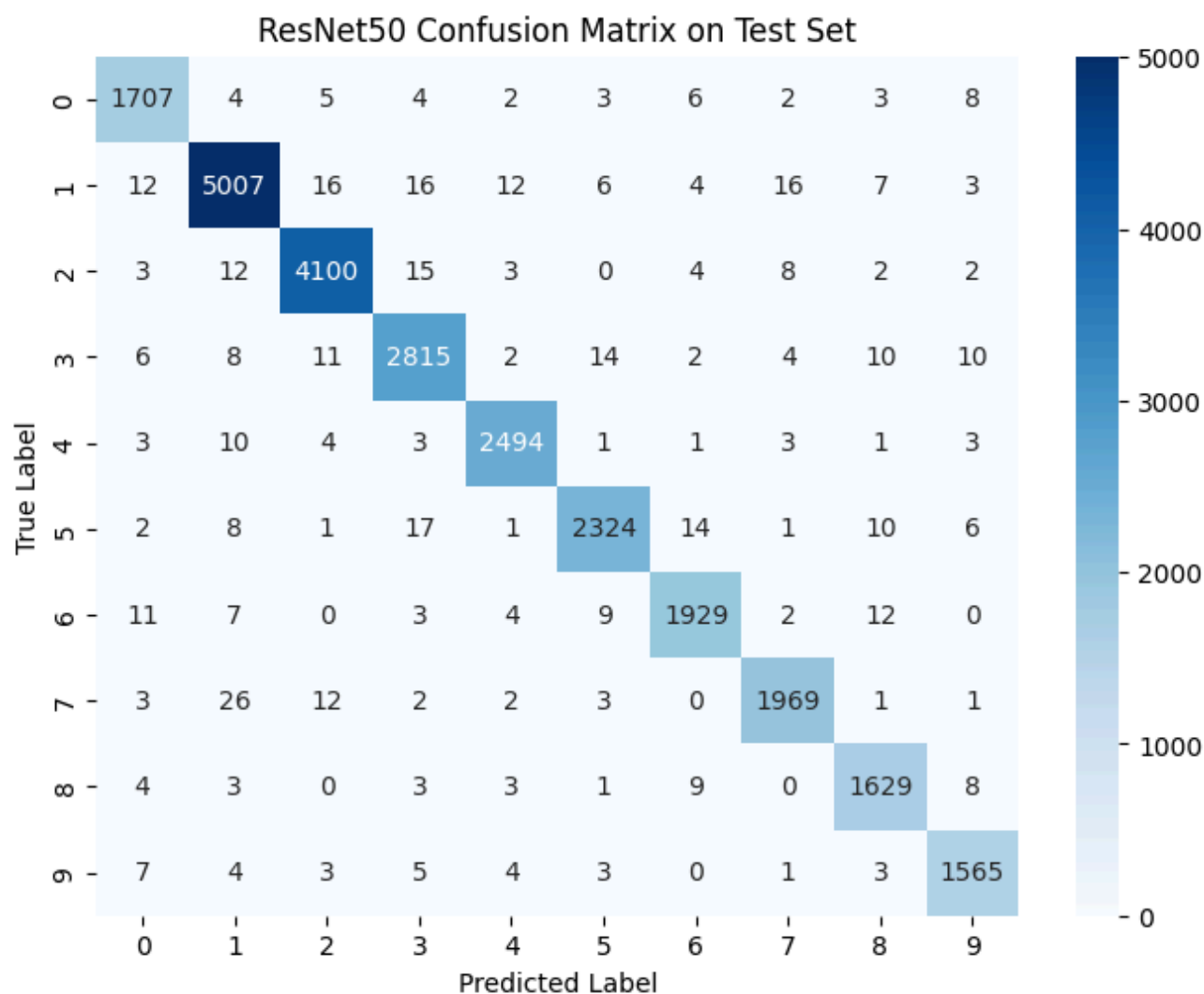
5.2.3.4 ResNet50 Evaluation on Test Dataset

After selecting the best-performing model based on validation performance, inference was performed on the unseen test dataset to assess the model's generalization capability. Final classification metrics, including accuracy, precision, recall, and F1-score, were computed to provide a comprehensive assessment of the model's effectiveness on previously unseen news articles.

ResNet50 Test Accuracy: 0.9811

ResNet50 Classification Report on Test Set:

	precision	recall	f1-score	support
0	0.97	0.98	0.97	1744
1	0.98	0.98	0.98	5099
2	0.99	0.99	0.99	4149
3	0.98	0.98	0.98	2882
4	0.99	0.99	0.99	2523
5	0.98	0.97	0.98	2384
6	0.98	0.98	0.98	1977
7	0.98	0.98	0.98	2019
8	0.97	0.98	0.98	1660
9	0.97	0.98	0.98	1595
accuracy			0.98	26032
macro avg	0.98	0.98	0.98	26032
weighted avg	0.98	0.98	0.98	26032



The confusion matrix above illustrates the performance of the ResNet50 model on the SVHN test set. Most digits are correctly classified, demonstrating that the model generalizes effectively to unseen data. Several digits are classified particularly well, with high numbers of correct predictions:

- Digit 1: 5007 correct predictions (highest)
- Digit 2: 4100 correct

- Digit 3: 2815 correct
- Digit 4: 2494 correct
- Digit 5: 2324 correct

This indicates that these digits have distinct visual characteristics, making them easier for the model to recognize. Some digits show slightly higher misclassification rates:

- Digit 8: 1629 correct, with confusion toward 6 and 9
- Digit 9: 1565 correct, with some confusion toward 0 and 3
- Digit 7: 1969 correct, with confusion toward 1 (26 cases)

The off-diagonal values reveal common confusion between visually similar digits:

- 1 ↔ 7:
 - 26 instances of 7 → 1
 - 16 instances of 1 → 7
- 3 ↔ 5:
 - 14 instances of 3 → 5
 - 17 instances of 5 → 3
- 8 ↔ 6:
 - 9 instances of 8 → 6
- 4 ↔ 1:
 - 10 instances of 4 → 1

Below is the accuracy of ResNet50 Model on every digit on the test set:

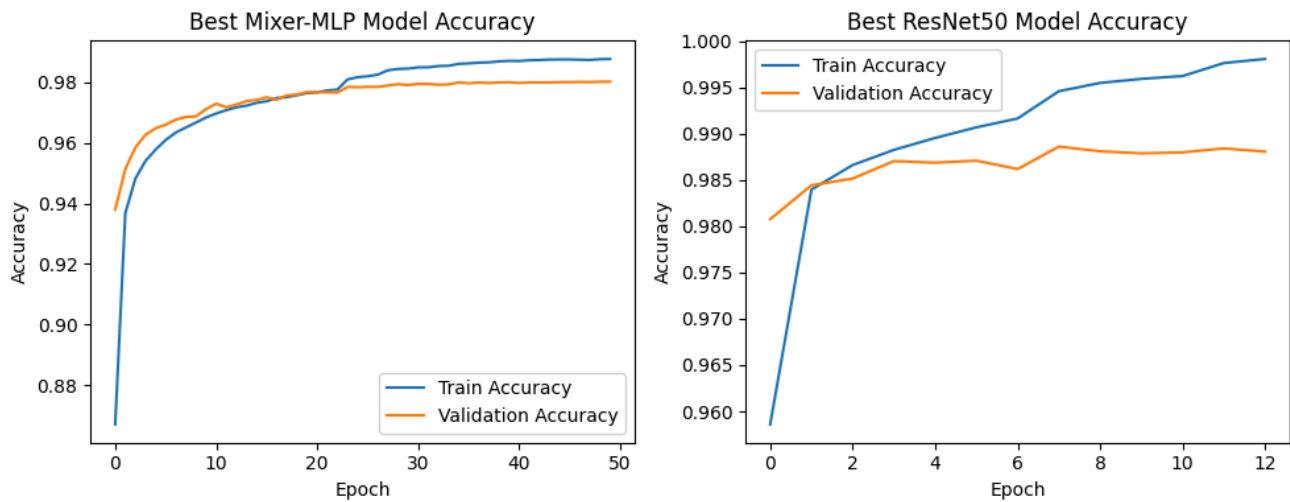
Per-Digit Accuracy of ResNet50 Model on Test Set:

```
-----
Digit 0: Accuracy = 97.88% (1707/1744)
Digit 1: Accuracy = 98.20% (5007/5099)
Digit 2: Accuracy = 98.82% (4100/4149)
Digit 3: Accuracy = 97.68% (2815/2882)
Digit 4: Accuracy = 98.85% (2494/2523)
Digit 5: Accuracy = 97.48% (2324/2384)
Digit 6: Accuracy = 97.57% (1929/1977)
Digit 7: Accuracy = 97.52% (1969/2019)
Digit 8: Accuracy = 98.13% (1629/1660)
Digit 9: Accuracy = 98.12% (1565/1595)
```

6. Model Evaluation and Performance Results

Comparison of Mixer-MLP vs ResNet50 Accuracy Plots.

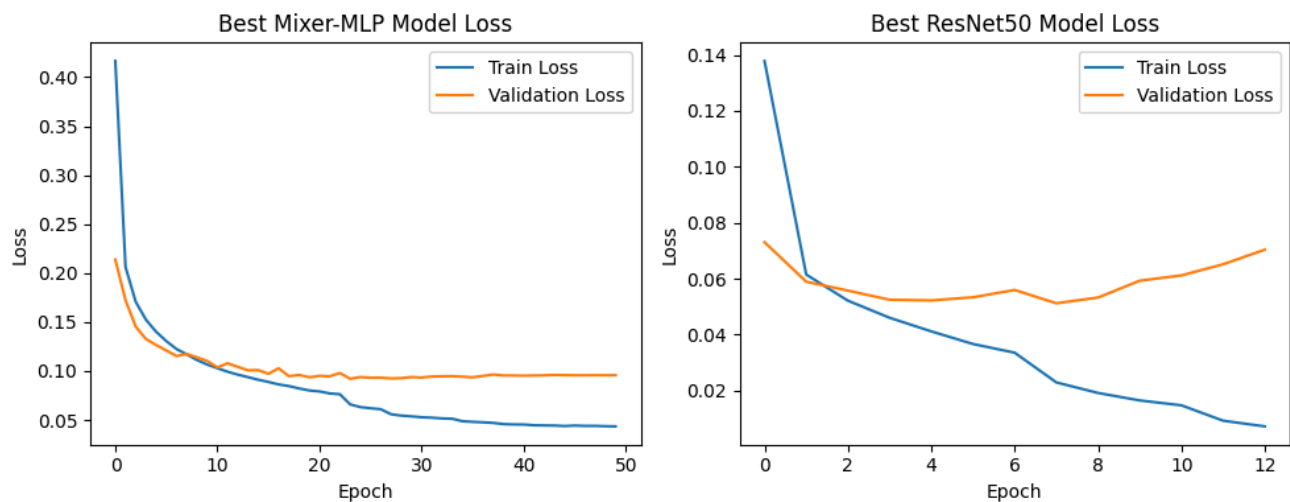
The two plots show the training and validation accuracy over epochs for the best-performing Mixer-MLP and ResNet50 models on the SVHN dataset.



From the above plots, it can be seen how much faster ResNet50's learning speed is compared to Mixer-MLP. Thanks to transfer learning (ImageNet), ResNet50 only needs a fewer epochs to learn, where in its final training accuracy, it came very close to reaching 100%. The final validation accuracy for Mixer-MLP was about ~98%, while ResNet50 is about ~98.8-99%. This means, the latter has better feature extraction. Mixer-MLP has shown a smaller gap on the train-validation accuracy, which means that it is not prone to overfitting hence more stable compared to ResNet50.

Comparison of Mixer-MLP vs ResNet50 Loss Plots.

The two plots show the training and validation loss over epochs for the best-performing Mixer-MLP and ResNet50 models on the SVHN dataset.



The loss curves of the Mixer-MLP model demonstrate stable and consistent learning throughout training. Both training and validation loss decrease smoothly over time, with the training loss steadily dropping from approximately 0.41 to around 0.04 and the validation loss from about 0.21 to roughly 0.09. The relatively small gap between the two curves indicates good generalization, as the model performs similarly on both seen and unseen data. Additionally, the absence of sharp fluctuations or increases in validation loss suggests that the model does not suffer from significant overfitting. The loss curves of the ResNet50 model indicate rapid learning and strong fitting to the training data, but also reveal signs of overfitting during later epochs. The training loss decreases sharply from approximately 0.14 to below 0.01 within a few epochs, demonstrating the model's ability to quickly learn discriminative features due to transfer learning. The validation loss initially decreases from

around 0.07 to about 0.05, showing early improvement in generalization; however, it begins to gradually increase after around 5-6 epochs. This widening gap between training and validation loss suggests that the model starts to overfit, as it continues to optimize for the training data while performance on unseen data deteriorates slightly.

Per-Digit Accuracy Comparison.

This comparison shows how accurately each model classified each digit from 0 to 9 on the test set. For each digit, the table reports:

- the accuracy percentage
- the number of correct predictions
- the total number of test samples for that digit

The **Cmp** column indicates which model performed better:

- < means ResNet50 performed better than Mixer-MLP
- > would mean Mixer-MLP performed better
- = would mean both performed the same

Per-Digit Accuracy Comparison on Test Set: Mixer-MLP vs ResNet50

Digit	Mixer-MLP	Cmp	ResNet50
0	97.25% (1696/1744)	<	97.88% (1707/1744)
1	97.02% (4947/5099)	<	98.20% (5007/5099)
2	97.37% (4040/4149)	<	98.82% (4100/4149)
3	95.77% (2760/2882)	<	97.68% (2815/2882)
4	97.03% (2448/2523)	<	98.85% (2494/2523)
5	95.55% (2278/2384)	<	97.48% (2324/2384)
6	95.60% (1890/1977)	<	97.57% (1929/1977)
7	96.43% (1947/2019)	<	97.52% (1969/2019)
8	93.67% (1555/1660)	<	98.13% (1629/1660)
9	95.67% (1526/1595)	<	98.12% (1565/1595)

From above table, ResNet50 clearly outperformed Mixer-MLP on every digit and achieved higher accuracy than Mixer-MLP for all 10 digit classes. This indicates that ResNet50 is consistently better at recognizing SVHN digits. Although ResNet50 was better across all classes, the Mixer-MLP still achieved high accuracy, mostly above 95%, and above 97% for some digits such as 0, 1, 2, and 4. This suggests that Mixer-MLP is a competitive model, but not as strong as ResNet50 for this image classification task. The biggest difference appears for:

- Digit 8:
 - Mixer-MLP: 93.67%
 - ResNet50: 98.13%
 - Difference: 4.46 percentage points
- Digit 9:
 - Mixer-MLP: 95.67%
 - ResNet50: 98.12%
 - Difference: 2.45 percentage points

This suggests that ResNet50 is much better at handling more visually challenging digits, especially those with shapes that may resemble other classes. ResNet50 exceeds 97.5% accuracy on every digit, and reaches nearly 99% on several classes, which indicates that the CNN-based model is highly effective at capturing the spatial patterns needed for robust digit recognition. Mixer-MLP struggles more on visually complex digits. Digits 3,4,6,8,9 are more difficult to recognize because of shape similarity, background clutter, blur, or variation in real-world house number images.

7. Discussion of Model Limitations or Constraints

Despite achieving strong performance, both the Mixer-MLP and ResNet50 models have several limitations. For the Mixer-MLP, its primary constraint lies in its limited ability to capture fine-grained spatial features, as it relies on fully connected layers rather than convolution. As a result, it performs slightly worse on visually complex digits, particularly those with similar shapes such as 8 and 9. Additionally, the model requires more training epochs to converge, making it less efficient in terms of training time. On the other hand, while ResNet50 achieves higher overall accuracy, it exhibits signs of overfitting, as seen from the divergence between training and validation loss. This suggests that the model may memorize training data patterns rather than fully generalizing to unseen samples. Furthermore, ResNet50 is computationally expensive due to its deep architecture and large input size (224×224), requiring greater memory and processing resources compared to Mixer-MLP. Another limitation shared by both models is the difficulty in distinguishing visually similar digits, which leads to consistent misclassification patterns across classes. In addition, resizing SVHN images from 32×32 to 224×224 for ResNet50 may introduce interpolation artifacts or redundant information, which could affect learning efficiency. Overall, while ResNet50 provides superior accuracy, it comes at the cost of higher computational complexity and a greater risk of overfitting, whereas Mixer-MLP offers a more stable and lightweight alternative but with slightly lower performance.

8. Potential Business Impact and Real-World Applications

The SVHN digit classification models, particularly ResNet50 and Mixer-MLP, have significant potential for real-world deployment across a range of business applications. One key application is in **automated address recognition systems**, where accurately detecting and classifying house numbers from street images can improve services such as navigation, logistics, and delivery routing. Companies in e-commerce and courier services can leverage such models to enhance delivery efficiency by automatically identifying addresses from images. In addition, these models can be applied in smart city infrastructure, **such as traffic monitoring systems**, where recognizing numeric information from road signs, vehicle plates, or building numbers can support urban planning and public safety. Financial institutions may also benefit from similar digit recognition systems in **document processing**, such as reading numeric information from forms, invoices, or checks, improving automation and reducing manual data entry errors. From a broader perspective, the strong

performance of ResNet50 highlights its suitability for high-accuracy, production-level systems, where reliability is critical, despite higher computational cost. In contrast, Mixer-MLP offers a more lightweight alternative, which may be beneficial for deployment in resource-constrained environments, such as mobile or edge devices, where computational efficiency and stability are important. Overall, the ability to accurately recognize digits in real-world images enables scalable automation across industries, leading to improved operational efficiency, reduced costs, and enhanced user experience.

9. Future Improvements / Future Work

Several enhancements can be explored to further improve the performance and robustness of the models used for SVHN digit classification. First, more advanced regularization techniques such as dropout tuning, weight decay optimization, or data augmentation strategies could help reduce overfitting, particularly for the ResNet50 model. Another potential improvement is experimenting with different model architectures, such as more recent CNN variants (e.g., EfficientNet) or hybrid architectures that combine convolutional layers with attention mechanisms. For the Mixer-MLP model, incorporating **patch-based improvements or hybrid CNN-MLP designs** could enhance its ability to capture spatial features more effectively. Further gains may also be achieved by using **larger or more diverse training data**, or applying advanced preprocessing techniques such as better normalization, contrast enhancement, or noise reduction. Hyperparameter optimization using methods like **grid search or Bayesian optimization** could also help identify more optimal configurations. Finally, future work could focus on **deployment considerations**, such as model compression, pruning, or quantization, to enable efficient inference on real-world systems, including mobile or edge devices. These improvements would enhance both the accuracy and practical usability of the models in real-world applications.

10. Conclusion

This project evaluated the effectiveness of two deep learning approaches—Mixer-MLP and ResNet50—for SVHN digit classification. The results show that both models are capable of achieving high performance, but they differ in learning behavior, efficiency, and overall effectiveness. The **Mixer-MLP model** demonstrated **stable and reliable training**, with smooth convergence and minimal overfitting. However, it required more epochs to reach optimal performance and showed slightly lower accuracy, particularly on more visually complex digits. In contrast, the **ResNet50 model** leveraged convolutional operations and transfer learning to achieve **faster convergence, lower loss, and consistently higher accuracy**. It outperformed Mixer-MLP across all digit classes and achieved a superior test accuracy of **98.11% compared to 96.37%**, confirming its effectiveness in capturing fine-grained spatial features such as edges and digit structures. However, it also showed signs of overfitting, indicating the need for careful regularization and early stopping. Overall, the findings suggest that **ResNet50 is the better choice for image recognition tasks**, particularly for real-world datasets like SVHN, due to its ability to learn rich hierarchical spatial features and deliver higher

classification accuracy. Meanwhile, Mixer-MLP remains a strong alternative when model simplicity, stability, or computational efficiency is a priority.

Appendix

A. ResNet50 Single-Sample Diagnostic Analysis

This section performs a diagnostic analysis on a single sample to understand how the ResNet50 model processes and learns from input data. It visualizes the original image alongside intermediate feature maps to illustrate how meaningful patterns are extracted across different layers. The transformation from extracted features to final predictions is examined through logits, loss, and gradients. In summary, this analysis offers a clearer view of the model's decision-making process and how its parameters are updated during training.

B. Hyperparameter Search and Model Training

The following procedure was used during hyperparameter optimization and training for all models used in this project:

1. Global Progress Tracking
 - Updated the global experiment state after each completed experiment, enabling interrupted hyperparameter searches to resume automatically from the last completed configuration.
2. Checkpoint Saving and Recovery
 - Saved the latest training state, best-performing model weights, hyperparameters, and training history to enable experiment recovery and reproducibility.
3. Experiment Resume Check
 - Checked for previously saved experiment checkpoints to determine whether training should resume from an earlier state or start from the beginning.
4. Hyperparameter Configuration
 - Iterated through each predefined hyperparameter combination specified in the search grid.
5. Data Loader Construction
 - Created training and validation data loaders using the selected batch size for the current experiment.
6. Model Initialization
 - Constructed the model architecture using the current hyperparameter configuration.
7. Training Component Setup
 - Constructed the model architecture using the current hyperparameter configuration.
8. Model Training
 - Trained the model on the training dataset for the specified number of epochs using mini-batch gradient descent.
9. Early Stopping
 - Terminated training when validation performance failed to improve for a predefined number of consecutive epochs, reducing the risk of overfitting and unnecessary computation.

10. Validation Evaluation and Experiment Result Storage

- Evaluated model performance on the validation dataset after each training epoch and recorded the best validation metrics and checkpoint information for each experiment to facilitate model comparison and final model selection.

11. Best Model Selection

- Compared validation performance across epochs and retained the model state corresponding to the highest validation accuracy.

C. Github Repository

This notebook, together with the needed files as well as saved checkpoints can be cloned from the following github repository and can be run on a computer. Please follow the `README.md` <https://github.com/yotsubakoiwai/tafe-ictss00120>

D. Google Colaboratory

Likewise, if system does not have sufficient disk space to save the necessary files, it can be run in [Google Colab](#)